

Layman's Guide: Building & Self-Hosting Your Voice AI Agent Platform

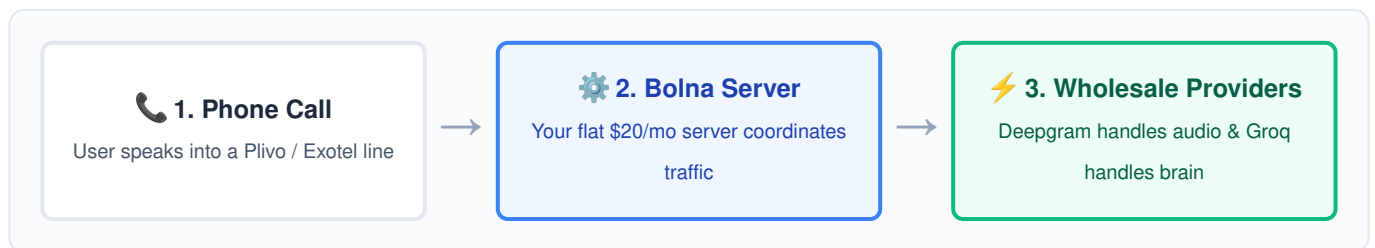
A Comprehensive, Visual 1-Week Step-by-Step Blueprint for a Team of 6 Interns

1. Introduction: What We Are Building & Why

Imagine giving your customers a web dashboard where they can type in a description of their business assistant, pick a voice, and get a working phone-based AI agent running in just **five minutes**. This guide explains how to build exactly that without paying expensive middleware costs.

Instead of buying commercial services that charge high per-minute middleman platform fees, you will use the open-source **Bolna** framework. By running this framework on your own cloud server (which costs a flat \$20 a month), your only variable expenses will be the absolute baseline wholesale utility costs for processing data.

Visual Framework: How Information Flows



This arrangement drops your aggregate operational usage cost down to ****under ₹0.80 per minute****. This allows you to offer highly disruptive, predictable monthly subscription pricing options to your end clients.

2. Phase 1: Building the Voice Agent Platform

Building the product means creating a web-based control panel (a frontend dashboard) where a client saves their settings, and building a backend database to save those settings and route them to your server engine.

STEP 2.1 — FRONTEND

Creating the User Input Control Panel

Your design team will build a clean web wizard interface. The layout needs three primary fields:

1. **The Agent Prompt Box:** A large text area where the business owner types out instructions (e.g., *"You are a real estate agent. Ask for the caller's budget and name."*).
2. **Language Selector Dropdown:** Allowing users to switch language configurations (like Indian English `en-IN` or Hindi `hi-IN`).
3. **Voice Selector Dropdown:** Letting the user choose standard male or female vocal tones.

STEP 2.2 — DATABASE

Structuring the Storage Architecture

When the user hits the "Deploy" button, your frontend captures their inputs and pushes them to a database server (like PostgreSQL). Here is the exact database format table structure to use:

```
CREATE TABLE custom_agents (  
  id SERIAL PRIMARY KEY,  
  agent_name VARCHAR(100) NOT NULL,  
  business_instructions TEXT NOT NULL,  
  chosen_language VARCHAR(10) DEFAULT 'en-IN',  
  chosen_voice VARCHAR(50) NOT NULL,  
  bolna_id VARCHAR(255)  
);
```

STEP 2.3 — INTEGRATION BRIDGE

Shipping the Configuration Code to the Bolna Engine

Your backend code will read the settings from the table structure and automatically packages it into a structured container. It then fires off an immediate web request to your live Bolna server:

```
import requests

def send_agent_configuration_to_bolna(agent_data):
    # This points directly to your self-hosted server's internal pipeline port
    bolna_api_url = "http://localhost:8001/agent"

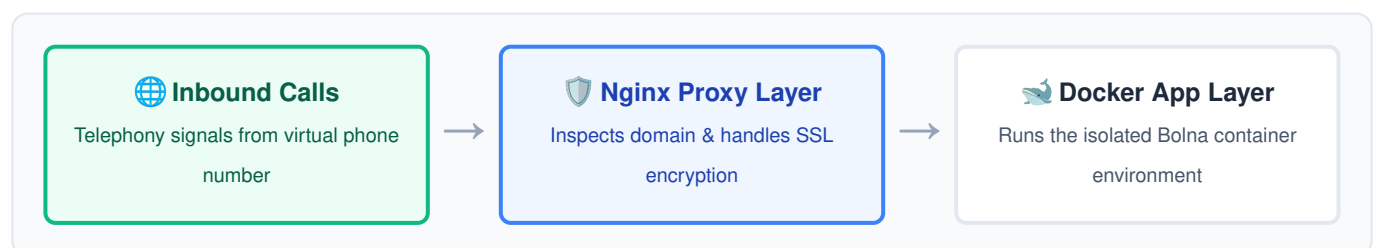
    # Organizing components for the voice pipeline
    structured_payload = {
        "assistant_config": {
            "assistant_name": agent_data["agent_name"],
            "tasks": [{
                "tools_config": {
                    "llm_config": {"provider": "openai", "model": "gpt-4o-mini", "max_tokens": 1000},
                    "asr_config": {"provider": "deepgram", "model": "nova-2", "language": agent_data["language"]},
                    "tts_config": {"provider": "deepgram", "voice": agent_data["chosen_voice"]}
                },
                "system_prompt": agent_data["business_instructions"]
            }]
        }
    }

    response = requests.post(bolna_api_url, json=structured_payload)
    return response.json().get("assistant_id")
```

3. Phase 2: How to Self-Host and Deploy for \$20/Month

Self-hosting simply means buying a blank virtual computer online, putting your open-source software onto it, and connecting it to the internet so it can accept real-time phone audio.

Visual Setup: Networking and Routing



STEP 3.1 — SERVER PROVISIONING

Setting Up the Virtual Server Node

Go to a hosting platform like DigitalOcean or AWS. Rent a virtual instance running **Ubuntu 24.04 LTS** with 2 CPUs and 4GB of RAM. This setup costs around \$15 to \$20 a month and can easily handle 20 to 30 phone conversations simultaneously. Point a custom domain or subdomain (e.g., `voice.yourportal.com`) to the server's public IP address.

STEP 3.2 — SERVER SETUP

Installing Software Packages via Terminal SSH

Log into your virtual computer using your command terminal and execute these commands step-by-step:

```
# Update the system software libraries
sudo apt update && sudo apt upgrade -y

# Install Docker container systems and web proxies
sudo apt install -y docker.io docker-compose git nginx

# Download the open-source Bolna source files
git clone https://github.com/bolna-ai/bolna.git
cd bolna/local_setup
```

STEP 3.3 — CONFIGURATIONS

Adding Wholesale Provider API Credentials

Open up the server's hidden settings file using the text editor: `nano .env`. Add your raw wholesale access keys directly into the configuration text file:

```
DEEPGRAM_API_KEY=your_actual_deepgram_api_key_here
OPENAI_API_KEY=your_actual_groq_or_openai_api_key_here
PORT=8001
```

Save and close the file, then boot up the entire system cluster container environment:

```
sudo ./start.sh
```

STEP 3.4 — NETWORK ROUTING

Configuring the Web Proxy Gateway

To securely stream real-time call sound waves into your server container, configure Nginx to route incoming connection requests smoothly. Open up a configuration matrix: `sudo nano /etc/nginx/sites-available/bolna` and paste the following parameters:

```
server {
    server_name voice.yourportal.com;

    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
    }
}
```

```
# Enable the web gateway and activate your proxy settings
sudo ln -s /etc/nginx/sites-available/bolna /etc/nginx/sites-enabled/
sudo systemctl restart nginx
```

Step 3.5 — Going Live with Telephony Lines

Log into an affordable telephony platform like **Plivo** or **Exotel**, rent a local virtual business number, and paste your new proxy web routing link into their incoming call handler box:

`ws://voice.yourportal.com/connection/incoming`

4. The 1-Week Action Plan for your 6 Interns

To build, configure, test, and deploy this complete platform within seven days, organize your team of six interns into three parallel development tracks:

Assigned Track	Core Task Responsibilities	Day-by-Day Implementation Goals
Interns 1 & 2 Frontend & UI Cell	• Build the 5-minute configuration form wizard. • Hook input forms up to your primary relational tables. • Build simple toggle buttons for accent profiles.	Days 1-2: Finish dashboard interface layouts. Days 3-4: Bind input variables to database tables. Days 5-7: Hook frontend events to backend API routers.
Interns 3 & 4 Backend & DevOps Cell	• Provision the cloud hosting server instances. • Install Docker environments and boot Bolna files. • Configure Nginx reverse proxies for WebSockets.	Days 1-2: Deploy Ubuntu VPS and launch Bolna containers. Days 3-4: Write the backend dynamic payload script wrappers. Days 5-7: Secure production domains and test stream routing.
Interns 5 & 6 Telephony Sync & QA	• Rent local virtual phone numbers on Plivo/Exotel consoles. • Link carrier webhooks directly to server proxy addresses. • Test conversations to verify low lag and smooth interruption handling.	Days 1-2: Establish sandbox phone testing lines. Days 3-4: Test payload formats for any system crashing issues. Days 5-7: Run end-to-end phone tests and launch the MVP live.